

# 作业 HW3 实验报告

姓名：庄子硕 学号：2350998 日期：2024 年 10 月 31 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

## 1. 涉及数据结构和相关背景

# 树和二叉树

## 2. 实验内容

### 3.1 二叉树的非递归遍历

#### 3.1.1 问题描述

二叉树的非递归遍历可通过栈来实现。例如对于由 `abc##d##ef###` 先序建立的二叉树，如下图 1 所示，中序非递归遍历（参照课本 p131 算法 6.3）可以通过如下一系列栈的入栈出栈操作来完成：`push(a) push(b) push(c) pop pop push(d)pop pop push(e) push(f) pop pop`。

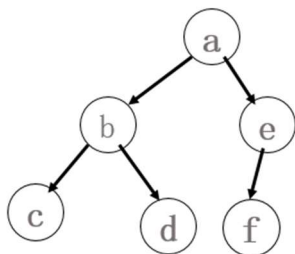


图 1

如果已知中序遍历的栈的操作序列，就可唯一地确定一棵二叉树。请编程输出该二叉树的后序遍历序列。

提示：本题有多种解法，仔细分析二叉树非递归遍历过程中栈的操作规律与遍历序列的关系，可将二叉树构造出来。

#### 3.1.2 基本要求

输入：第一行一个整数  $n$ ，表示二叉树的结点个数。

接下来  $2n$  行，每行描述一个栈操作，格式为：`push X` 表示将结点  $X$  压入栈中，`pop` 表示从栈中弹出一个结点。

( $X$  用一个字符表示)

输出：一行，后序遍历序列。

#### 3.1.3 数据结构设计

```
struct Node {
```

```

    char elem=0;
    int lchild;
    int rchild;
};

```

### 3.1.4 功能说明（函数、类）

# 推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

# 不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

# 例如

//后序遍历打印树

```

void PostTraverse(int node)
{
    if (list[node].elem == 0)
        return;

    PostTraverse(list[node].lchild);
    PostTraverse(list[node].rchild);

    if(list[node].elem!=1)//不打印头结点
        printf("%c", list[node].elem);
}

```

//遍历建立树

```

while (cin.getline(inp, 100)&&cnt)
{
    if (strncmp(inp, "push", 4) == 0)
    {
        //出栈
        int cur=--top;

        list[stack[cur]].elem =inp[5];
        list[stack[cur]].lchild = Endlist++;
        list[stack[cur]].rchild = Endlist++;

        int lchild = list[stack[cur]].rchild;
        int rchild = list[stack[cur]].lchild;
        //入栈
        stack[top++] = lchild;
        stack[top++] = rchild;

        cnt--;
    }
    else if (strncmp(inp, "pop", 3) == 0)
    {

```

```

    top--;
    //如果 p 是左，则更新 p 为右
    //p 为右，更新 p 为父的父
    //当左右均为 NULL，无法判断
}

```

### 3.1.5 调试分析（遇到的问题和解决方法）

**问题 1：不理解遍历过程；解决方法 1：看书**

### 3.1.6 总结和体会

# 可以说说做此题的收获、分析这道题目的难点和易错点（数据边界、对算法效率的要求等）

- (1) 注意标记头结点
- (2) 注意用数组实现栈时，栈顶指针的移动

## 3.2 二叉树的同构

### 3.2.1 问题描述

给定两棵树 T1 和 T2。如果 T1 可以通过若干次左右孩子互换变成 T2，则我们称两棵树是“同构”的。例如图 1 给出的两棵树就是同构的，因为我们把其中一棵树的结点 a、b、e 的左右孩子互换后，就得到另外一棵树。而图 2 就不是同构的。

外一棵树。而图 2 就不是同构的。

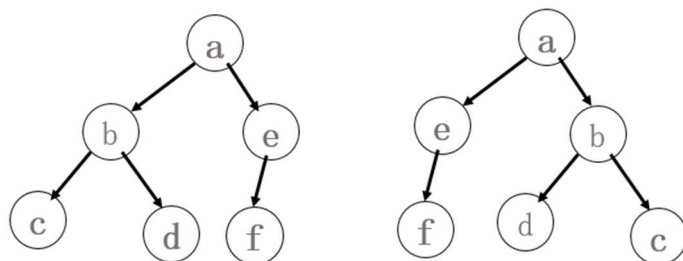


图 1

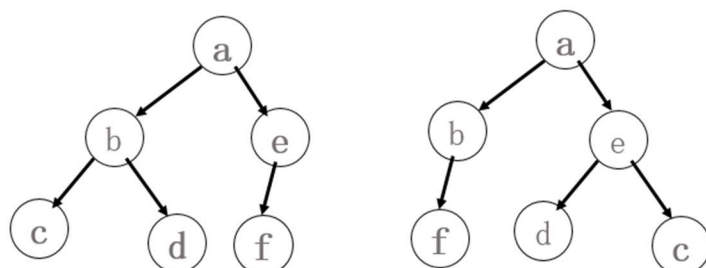


图 2

现给定两棵树，请你判断它们是否是同构的。并计算每棵树的深度。

### 3.2.2 基本要求

输入：

第一行是一个非负整数 N1，表示第 1 棵树的结点数；

随后 N 行，依次对应二叉树的 N 个结点（假设结点从 0 到 N-1 编号），每行有三项，分别是 1 个英文大写字母、其左孩子结点的编号、右孩子结点的编号。如果孩子结点为空，则在相应位置上给出“-”。给出的数据间用一个空格分隔。

接着一行是一个非负整数 N2，表示第 2 棵树的结点数；

随后 N 行同上描述一样，依次对应二叉树的 N 个结点。

输出：

共三行。

第一行，如果两棵树是同构的，输出“Yes”，否则输出“No”。

后面两行分别是两棵树的深度。

### 3.2.3 数据结构设计

```
struct Node {
    char elem=0;
    int lchild=-1;
    int rchild=-1;
};
```

### 3.2.4 功能说明（函数、类）

//递归判断是否同构

```
bool is_isomorphic(int root1, int root2)
{
    if (list[root1].elem == 0 && list2[root2].elem == 0)
        return true;
    if (list[root1].elem != list2[root2].elem)
        return false;
    return (is_isomorphic(list[root1].lchild, list2[root2].lchild) &&
is_isomorphic(list[root1].rchild, list2[root2].rchild)) ||
        (is_isomorphic(list[root1].rchild, list2[root2].lchild) &&
is_isomorphic(list[root1].lchild, list2[root2].rchild));
}
```

### 3.2.5 调试分析（遇到的问题 and 解决方法）

问题 1：oj 给出的输入描述模糊不清，导致错误的读取数据方式；

解决方案 1：多试，多问；

### 3.2.6 总结和体会

出题人应当尽量清晰地描述题目要求，作答者才能更好更方便地完成题目

### 3.3 感染二叉树的总时间

#### 3.3.1 问题描述

给你一棵二叉树的根节点 `root`，二叉树中节点的值互不相同。另给你一个整数 `start`。在第 0 分钟，感染将会从值为 `start` 的节点开始爆发。

每分钟，如果节点满足以下全部条件，就会被感染：

节点此前还没有感染。

节点与一个已感染节点相邻。

返回感染整棵树需要的分钟数。

#### 3.3.2 基本要求

输入

第一行包含两个整数 `n` 和 `start`

接下来包含 `n` 行，描述 `n` 个节点的左、右孩子编号

0 号节点为根节点， $0 \leq \text{start} < n$

对于 20% 的数据， $1 \leq n \leq 10$

对于 40% 的数据， $1 \leq n \leq 1000$

对于 100% 的数据， $1 \leq n \leq 100000$

下载编译并运行 `p128_data.cpp` 生成随机测试数据

输出

一个整数，表示感染整棵二叉树所需要的时间

#### 3.3.3 数据结构设计

```
struct Node {
    int parent = -1;
    int lchild = -1;
    int rchild = -1;
    short flag = 0;
};
```

#### 3.3.4 功能说明（函数、类）

```
int mins = 0;
int cnt = 1;
int JustInfected = 1;
```

```
My_Queue<int> q1;
q1.push(SrcNum);
list[SrcNum].flag = 1;
```

```
//用栈存储当前最外围感染者
while (cnt < num)
```

```

{
    int CurInfect = 0;
    for (int i = 0; i < JustInfected; i++)
    {
        int infect = q1.front();
        q1.pop();

        if (list[infect].lchild != -1 && list[list[infect].lchild].flag == 0)
        {
            q1.push(list[infect].lchild);
            CurInfect++;
            list[list[infect].lchild].flag = 1;
        }

        if (list[infect].rchild != -1 && list[list[infect].rchild].flag == 0)
        {
            q1.push(list[infect].rchild);
            CurInfect++;
            list[list[infect].rchild].flag = 1;
        }

        if (list[infect].parent != -1 && list[list[infect].parent].flag == 0)
        {
            q1.push(list[infect].parent);
            CurInfect++;
            list[list[infect].parent].flag = 1;
        }
    }

    JustInfected = CurInfect;
    mins++;
    cnt += JustInfected;
}

printf("%d", mins);

```

### 3.3.5 调试分析（遇到的问题 and 解决方法）

问题 1: oj 给出的输入描述模糊不清，导致错误的读取数据方式；

解决方案 1: 多试，多问

### 3.3.6 总结和体会

出题人应当尽量清晰地描述题目要求，作答者才能更好更方便地完成题目

## 3.4 树的重构

### 3.4.1 问题描述

树木在计算机科学中有许多应用。也许最常用的树是二叉树,但也有其他的同样有用的树类型。其中一个例子是有序树(任意节点的子树是有序的)。

每个节点的子节点数是可变的,并且数量没有限制。一般而言,有序树由有限节点集合  $T$  组成,并且满足:

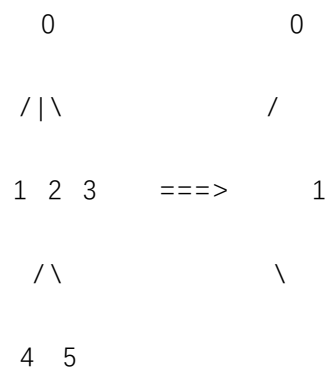
1. 其中一个节点置为根节点, 定义为  $\text{root}(T)$ ;
2. 其他节点被划分为若干子集  $T_1, T_2, \dots, T_m$ , 每个子集都是一个树。

同样定义  $\text{root}(T_1), \text{root}(T_2), \dots, \text{root}(T_m)$  为  $\text{root}(T)$  的孩子, 其中  $\text{root}(T_i)$  是第  $i$  个孩子。节点  $\text{root}(T_1), \dots, \text{root}(T_m)$  是兄弟节点。

通常将一个有序树表示为二叉树是更加有用的, 这样每个节点可以存储在相同内存空间中。有序树到二叉树的转化步骤为:

1. 去除每个节点与其子节点的边
2. 对于每一个节点, 在它与第一个孩子节点(如果存在)之间添加一条边, 作为该节点的左孩子
3. 对于每一个节点, 在它与下一个兄弟节点(如果存在)之间添加一条边, 作为该节点的右孩子

如图所示:



在大多数情况下, 树的深度(从根节点到叶子节点的边数的最大值)都会在转化后增加。这是不希望发生的事情因为很多算法的复杂度都取决于树的深度。

现在，需要你实现一个程序来计算转化前后的树的深度。

### 3.4.2 基本要求

输入：由多行组成，每一行都是一棵树的深度优先遍历时的方向。其中 d 表示下行(down)，u 表示上行(up)。

例如上面的树就是 d u d d u d u d u，表示从 0 下行到 1, 1 上行到 0, 0 下行到 2 等等。输入的截止为以 # 开始的行。

可以假设每棵树至少含有 2 个节点，最多 10000 个节点。

输出：对每棵树，打印转化前后的树的深度，采用以下格式 Tree t: h1 => h2。其中 t 表示样例编号(从 1 开始)，h1 是转化前的树的深度，h2 是转化后的树的深度。

### 3.4.3 数据结构设计

```
struct Node {
    TElemType data;
    int leftChild = -1, rightChild = -1;
};
```

```
typedef Node Bitree[N];
Bitree btree;
```

```
typedef Node* SElemType;
struct SqStack {
    SElemType* base;
    SElemType* top;
    int stacksize;
};
```

### 3.4.4 功能说明（函数、类）

//返回二叉树的深度

```
int Depth(Bitree T, Bitree& bt) {
    if (T->leftChild == -1 && T->rightChild == -1) return 1;
    int leftDepth = T->leftChild != -1 ? Depth(bt + T->leftChild, bt) : 0;
    int rightDepth = T->rightChild != -1 ? Depth(bt + T->rightChild, bt) : 0;
    return max(leftDepth, rightDepth) + 1;
}
```

### 3.4.5 调试分析（遇到的问题 and 解决方法）

问题 1：用递归遍历有序树的方式复杂度太高

解决方案 1：在同等渐进复杂度下，用系数更小的方式：建立并遍历二叉树

### 3.4.6 总结和体会

能用简单的操作完成的，就不要用过于复杂的操作

## 3.5 最近公共祖先

### 3.5.1 问题描述

给出一颗多叉树，请你求出两个节点的最近公共祖先。

一个节点的祖先节点可以是该节点本身，树中任意两个节点都至少有一个共同祖先，即根节点。

### 3.5.2 基本要求

输入：

输入数据包含  $T$  个测试样本，每个样本  $i$  包含  $N_i$  个节点和  $N_i-1$  条边和  $M_i$  个问题，树中节点从 1 到  $N_i$  编号

输入第一行是测试样本数  $T$

每个测试样本  $i$  第一行为两个整数  $N_i$  和  $M_i$

接下来  $N_i-1$  行，每行 2 个整数  $a$ 、 $b$ ，表示  $a$  是  $b$  的父节点

接下来  $M_i$  行，每行两个整数  $x$ 、 $y$ ，表示询问  $x$  和  $y$  的共同祖先

对于 100% 的数据，

$1 \leq T \leq 100$ ;

$5 \leq N \leq 1000$ ;

$5 \leq M \leq 1000$ ;

输出：

对于每一个询问输出一个整数，表示共同祖先的编号

### 3.5.3 数据结构设计

```
struct Node {  
    int parent = -1;  
    vector<int> SubNodes;  
};
```

### 3.5.4 功能说明（函数、类）

//先从一个节点开始向上遍历，再从另一个节点开始

```
for (int i = 0; i < M; i++)  
{  
    memset(parent, 0, sizeof(int) * N+1);
```

```

int kid1, kid2;
scanf("%d%d", &kid1, &kid2);

//从 kid1 向上遍历，记录经历的 parent
int cur = kid1;
while (1)
{
    parent[cur] = 1;
    if (list[cur].parent != -1)
        cur = list[cur].parent;
    else
        break;
}

cur = kid2;
while (cur != -1 && !parent[cur])
{
    cur = list[cur].parent;
}

printf("%d\n", cur);
}

```

### 3.5.5 调试分析（遇到的问题 and 解决方法）

问题 1：如何存储多子树

解决方案 1：用 vector

问题 2：TLE

解决方案 2：清除状态时智慧一点；数组不要开太大

### 3.5.6 总结和体会

注意参数输入格式，

## 3.6 求树的后序

### 3.6.1 问题描述

给出二叉树的前序遍历和中序遍历，求树的后序遍历

### 3.6.2 基本要求

输入包含若干行，每一行有两个字符串，中间用空格隔开

同行的两个字符串从左到右分别表示树的前序遍历和中序遍历，由单个字符组成，每个字符表示一个节点

字符仅包括大小写英文字母和数字，最多 62 个

输入保证一颗二叉树内不存在相同的节点

### 3.6.3 数据结构设计

```
struct Node {
    int lchild = -1;
    int rchild = -1;
};
```

### 3.6.4 功能说明（函数、类）

//根据前后序遍历，给出二叉树：标志位不清晰：0 表示空节点，-1 表示非法

```
int giveBinaryTree(char* ForTraPos,char* MidTraPos,int SeqTraLen)
{
    if (SeqTraLen == 0)
        return 0;
    if (SeqTraLen == 1)
    {
        if (ForTraPos[0] == MidTraPos[0])
        {
            list[MidTraPos[0]].lchild = 0;
            list[MidTraPos[0]].rchild = 0;
            return MidTraPos[0];
        }
        return -1;
    }
    char* RootInMid=NULL;
    //这里是问题的根本：搜索越界，原来是 strchr
    //所以能跑不能说明任何问题，只能说编译器优化的真好
    for (int i = 0; i < SeqTraLen; i++)
        if (*(MidTraPos + i) == ForTraPos[0])
        {
            RootInMid = MidTraPos + i;
            break;
        }
    RootInMid = strchr(MidTraPos, ForTraPos[0]);

    if (RootInMid == NULL)//没找到
        return -1;

    int NextForTraLen = RootInMid - MidTraPos;//这个一定为正
    int NextPostTraLen = SeqTraLen - NextForTraLen - 1;//这个可能为负，从而死
    循环

    int lc=giveBinaryTree(ForTraPos+1, MidTraPos, NextForTraLen);
    int      rc=giveBinaryTree(ForTraPos+1+NextForTraLen,      MidTraPos+1+
```

```
NextForTraLen, NextPostTraLen);
```

```
    if (lc != -1 && rc != -1)
    {
        list[(int)ForTraPos[0]].lchild = lc;
        list[(int)ForTraPos[0]].rchild = rc;

        return ForTraPos[0];
    }
    else
        return -1;
}
```

//后序遍历树

```
void PostTraverse(int root)
{
    if(list[root].lchild!=0)
        PostTraverse(list[root].lchild);
    if(list[root].rchild!=0)
        PostTraverse(list[root].rchild);

    if(isalnum(root))
        printf("%c", root);
}
```

### 3.6.5 调试分析（遇到的问题解决方法）

问题 1：本地没问题，oj 上 RE

解决方案 1：排查逻辑漏洞，发现遍历越界

### 3.6.6 总结和体会

本地能跑！=真的没问题；测试数据很重要；要逐步 debug

## 3.7 表达式树

### 3.7.1 问题描述

任何一个表达式，都可以用一棵表达式树来表示。例如，表达式  $a+b*c$ ，可以表示为如下的表达式树：

+

```

/\
a  *
  /\
  b c

```

现在，给你一个中缀表达式，这个中缀表达式用变量来表示（不含数字），请你将这个中缀表达式用表达式二叉树的形式输出出来。

### 3.7.2 基本要求

输入分为三个部分。

第一部分为一行，即中缀表达式(长度不大于 50)。

中缀表达式可能含有小写字母代表变量（a-z），也可能含有运算符（+、-、\*、/、小括号），不含有数字，也不含有空格。

第二部分为一个整数  $n$  ( $n \leq 10$ )，表示中缀表达式的变量数。

第三部分有  $n$  行，每行格式为  $C \quad x$ ， $C$  为变量的字符， $x$  为该变量的值。

输出分为三个部分，第一个部分为该表达式的逆波兰式，即该表达式树的后根遍历结果。占一行。

第二部分为表达式树的显示，如样例输出所示。

如果该二叉树是一棵满二叉树，则最底部的叶子结点，分别占据横坐标的第 1、3、5、7……个位置（最左边的坐标是 1），

然后它们的父结点的横坐标，在两个子结点的中间。

如果不是满二叉树，则没有结点的地方，用空格填充（但请略去所有的行末空格）。

每一行父结点与子结点中隔开一行，用斜杠 (/) 与反斜杠 (\) 来表示树的关系。

/出现的横坐标位置为父结点的横坐标偏左一格，\出现的横坐标位置为父结点的横坐标偏右一格。

也就是说，如果树高为  $m$ ，则输出就有  $2m-1$  行。

第三部分为一个整数，表示将值代入变量之后，该中缀表达式的值。需要注意的一点是，除法代表整除运算，即舍弃小数点后的部分。

同时，测试数据保证不会出现除以 0 的现象。

### 3.7.3 数据结构设计

```

struct Node {
    char VarsName = 0;
    int VarsValue = 0;
    int lchild = -1;
    int rchild = -1;
}

```

### 3.7.4 功能说明（函数、类）

//返回 rchar 入栈与否的真值

```

bool Compare(char lchar, char rchar)
{
    //括号是右结合的
    if (lchar == '(' && rchar == ')')

```

```

        return true;

    if (lchar == ')' || rchar == '(')
        return false;
    if (rchar == '(' || lchar == ')')
        return true;

    if ((lchar == '+' || lchar == '-') && (rchar == '*' || rchar == '\\'))
        return true;

    if ((rchar == '+' || rchar == '-') && (lchar == '*' || lchar == '\\'))
        return false;

    return false;
}

```

//根据输入的运算符计算

```

void Caculate(char calculator)
{
    if (calculator == ')')
    {
        if (SignStack[SignTop - 1] != '(')
        {
            Caculate(SignStack[--SignTop]);
            SignStack[++SignTop] = ')';
        }
        else
            SignTop--;
    }
    else
    {
        int var2 = VarStack[--VarsTop];
        int var1 = VarStack[--VarsTop];

        Node& CRoot = list[Endlist];

        CRoot.lchild = var1;
        CRoot.rchild = var2;

        int CurCHeight = max(TreeHeight[var1], TreeHeight[var2]) + 1;
        int &OriCHeight = TreeHeight[Endlist];
        OriCHeight = CurCHeight > OriCHeight ? CurCHeight : OriCHeight;

        if (Root == var1 || Root == var2)

```

```

        Root = Endlist;

    CRoot.VarsName = calculator;
    if (calculator == '+')
        CRoot.VarsValue = list[var1].VarsValue + list[var2].VarsValue;
    else if (calculator == '-')
        CRoot.VarsValue = list[var1].VarsValue - list[var2].VarsValue;
    else if (calculator == '*')
        CRoot.VarsValue = list[var1].VarsValue * list[var2].VarsValue;
    else if (calculator == '/')
        CRoot.VarsValue = (int)((double)list[var1].VarsValue /
(double)list[var2].VarsValue);

    VarStack[VarsTop++] = Endlist++; //在任务的最后再用++, 免得访问错误
}
}

```

//给出 list 中根节点的编号, 打印后序遍历结果

```

void PostTraverse(int root)
{
    if (list[root].lchild != -1)
        PostTraverse(list[root].lchild);
    if (list[root].rchild != -1)
        PostTraverse(list[root].rchild);

    printf("%c", list[root].VarsName);
}

```

//打印表达式树

```

void PrintGraph(int Depth, int spaces)
{
    if (Depth < 1)
        return;

    int MaxLeaves = pow(2, Depth - 1);

    PrintGraph(Depth - 1, 2 * spaces + 1);

    if (Depth != 1)
    {
        //行间字符

        for (int i = 0; i < 2 * spaces ; i++)
            cout << ' ';
    }
}

```

```

    for (int i = 0; i < MaxLeaves/2; i++)
    {
        if (Graph[Depth - 1][2 * i] != -1)
            cout << '/';
        else
            cout << ' ';

        cout << ' ';

        if (Graph[Depth - 1][2 * i + 1] != -1)
            cout << '\\';
        else
            cout << ' ';

        if(i<MaxLeaves/2-1)
            for (int j = 0; j < 4 * spaces + 1; j++)
                cout << ' ';
    }

    cout << endl;
}
for (int i = 0; i < spaces; i++)
    cout << ' ';

for (int i = 0; i < MaxLeaves; i++)
{
    char Cont = (Graph[Depth - 1][i] != -1) ? list[Graph[Depth -
1][i]].VarsName : ' ';
    cout <<Cont;

    if (i < MaxLeaves-1)
        for (int j = 0; j < spaces * 2 + 1; j++)
        {
            cout << ' ';
        }
}

cout << endl;
}

```

### 3.7.5 调试分析（遇到的问题解决方法）

问题 1: 如何找到根节点

解决方案 1: 从任意一个节点开始, 有父节点则更新根节点为父节点

问题 2: 如何生成树

解决方案 2: 在用栈进行表达式求值的同时, 进行结点的组合 (表达式求值的结果可以放在新节点中)

问题 3: 本地运行没问题, oj 运行 WA

解决方案 3: 检查发现, 非常巧妙, 计算优先级时, 仅当前加减后乘除时出现问题, 而数据中要么用()围起来, 要么先乘除后加减, 从而只有这一个数据是先加减后乘除

### 3.7.6 总结和体会

不要给自己找麻烦, 解决题目是最重要的

## 3. 实验总结

不要死磕, 当一个项目已经想过了, 但是没有进展, 就开始下一个项目。